# Data Structure 1

Paul Blondel

CNAM

September 1st, 2018

*'I will, in fact, claim that the difference between a bad programmer and a good one is whether he considers his code or his data structures more important. Bad programmers worry about the code. Good programmers worry about data structures and their relationships.'*

Linus Torvalds

1. Data
   - Data, types and properties
   - Units
2. Memory
   - Different memories for different usage
   - How it works?
3. Algorithm
   - What is an algorithm?
   - Example of algorithm
   - Well designing an algorithm is important!
   - Evaluating the complexity
   - Example of complexity calculus
   - Complexity definitions
   - The big O notation
   - Tips to find the complexity
   - Examples of time and memory consumption estimations
4. Conclusion

## What is data?

Data is the result of a measurement

Examples: The diameter of the sun is **1.39 million kilometers**, the temperature of the room is **30°C**, and so on.

A data ...

- ... is **simple** or **complex** (composed of simple data)
- ... has a **type** (integer, character, and so on)
- ... has a set of authorized **instructions**

Examples: 'hello' is a complex data (composed of simple character data), 30°C is a simple integer data. 'hello' * 30 is not authorized, but 2 * 30 is.

The most common units are:

- 1 bit (0 or 1)
- 1 byte (or octet) = 8 bits
- 1 KB (or KO) = 1000 bytes[1]
- 1 MB (or MO) = $10^6$ bytes = 1000 KB
- 1 GB (or GO) = $10^9$ bytes = $10^6$ KB = 1000 MB
- 1 TB (or TO) = $10^{12}$ bytes

Exemple: 'hello' is stored on 5 bytes (each character is stored on 1 byte).

[1] 1 KB is sometimes defined as equal to 1024 bytes, but this definition is no longer valid. In 1998, the International Electrotechnical Commission stated that 1 KB = 1000 bytes and that 1024 bytes = 1 KiB (kibibyte).

Two main types[2] of data storage:

- The main memory (Random-access memory or RAM)
  - Storing running softwares and opened documents
- The mass storage memory
  - Storing installed softwares and all documents
  - Two main technologies on the market:
    - ★ Hard-disk drive (HDD) (invented in 1954)
    - ★ Solid-state drive (SSD) (invented in 1978)

| | RAM | HDD / SSD |
|---|---|---|
| Access speed | Fast (2000-26600MO/s) | Slow (50-500MO/s) |
| Storage capacity | Low (4-32GB) | High (256GB - 1TB) |
| Storage property | Volatile | Not Volatile |

[2]The CPU cache is another interesting memory, it stores copies of data frequently fetched from the main memory. Interesting fact: Graphics Processing Units dedicated to computing can have up to 5 different data storage types.

- RAM
  - All the data is stored in an array of memory cells
  - 1 bit = 1 memory cell (DRAM: bit stored in a capacitor)

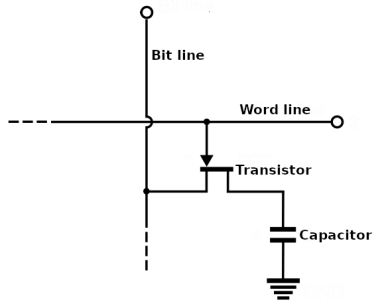



Figure: DRAM memory cell

- HDD
  - All the data is stored in disks of ferromagnetic material
  - 1 bit = direction of magnetic field lines

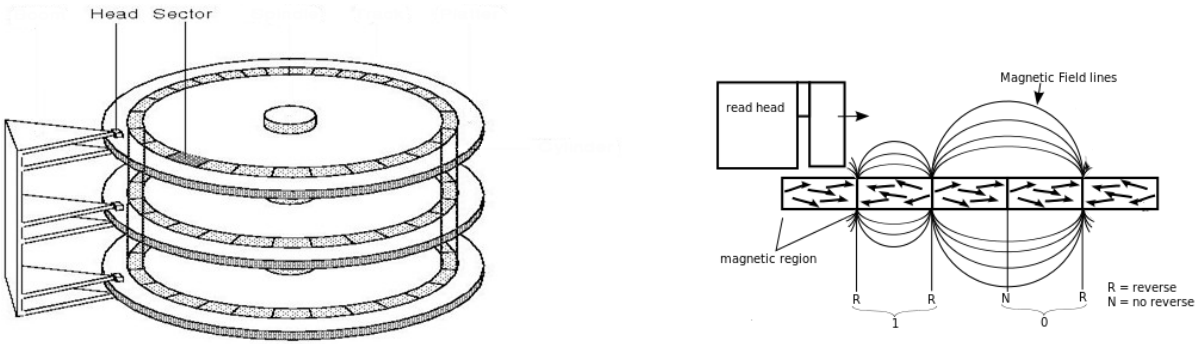



Figure: Left: disks of ferromagnetic material right: magnetic field lines

- SSD
  - All the data is stored in an array of floating-gate transistors
  - No mechanical moving component = faster

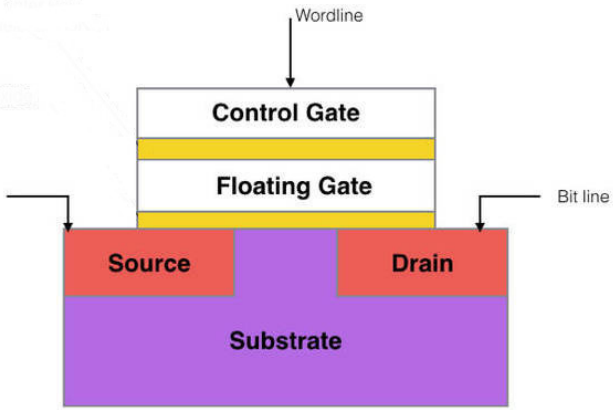



Figure: Floating gate transistor cell

1. Data
   - Data, types and properties
   - Units
2. Memory
   - Different memories for different usage
   - How it works?
3. Algorithm
   - What is an algorithm?
   - Example of algorithm
   - Well designing an algorithm is important!
   - Evaluating the complexity
   - Example of complexity calculus
   - Complexity definitions
   - The big O notation
   - Tips to find the complexity
   - Examples of time and memory consumption estimations
4. Conclusion

## What is an algorithm?

An algorithm is a **finite sequence of instructions** designed to **solve a specific problem**. An algorithm is like a cooking recipe.

Writing an algorithm is a preliminary step before coding:

- All the required instructions are listed and detailed
- The coherence of the sequence can be checked
- The sequence of instructions can be optimized
- Time and memory consumption can be estimated and optimized

## Pseudo-code

Algorithms are often (but not always) written in 'pseudo-code'.
Pseudo-code is a high-level intuitive pseudo-language with no reference to existing programming languages and no specific standard.

## Example of algorithm: finding the Greatest Common Divisor (GCD)

Euclid said: the GCD of two numbers divides their difference.

Let's write the appropriate algorithm in pseudo-code ...

## Example of algorithm: finding the Greatest Common Divisor (GCD)

Euclid said: the GCD of two numbers divides their difference.

Let's write the appropriate algorithm in pseudo-code ...

```
Data: a and b
gcd = 1;
while not a*b == 0 do
    if a>b then
        a = a-b;
    else
        b = b-a;
    end
end
if a==0 then
    gcd = b;
else
    gcd = a;
end
```

**Algorithm 2:** Euclid's algorithm in pseudo-code

An algorithm with an inappropriate design can ...

- ... take too much time to execute (several hours to several years!)
- ... take too much memory (Earth's storage memory is not enough!)

An algorithm with an inappropriate design can ...

- ... take too much time to execute (several hours to several years!)
- ... take too much memory (Earth's storage memory is not enough!)

**Where does it come from?**

Most of the time it is due to a **too simple** or **too naïve approach of the problem**. Sometimes it's because the **memory management** and/or **the sequence of instructions** are not optimized well enough.

An algorithm with an inappropriate design can ...

- ... take too much time to execute (several hours to several years!)
- ... take too much memory (Earth's storage memory is not enough!)

**Where does it come from?**

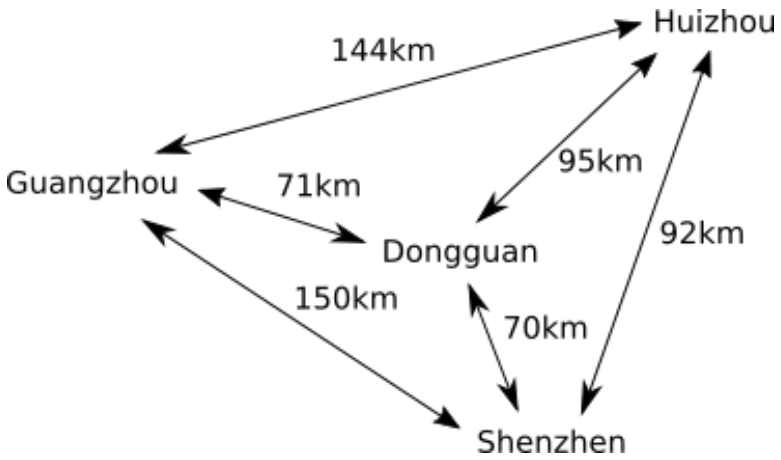
Most of the time it is due to a **too simple** or **too naïve approach of the problem**. Sometimes it's because the **memory management** and/or **the sequence of instructions** are not optimized well enough.

**How can I know if my algorithm design is appropriate enough?**

By evaluating the **time complexity** and/or the **space complexity**!

## Example: the travelling salesman problem (naïve approach)

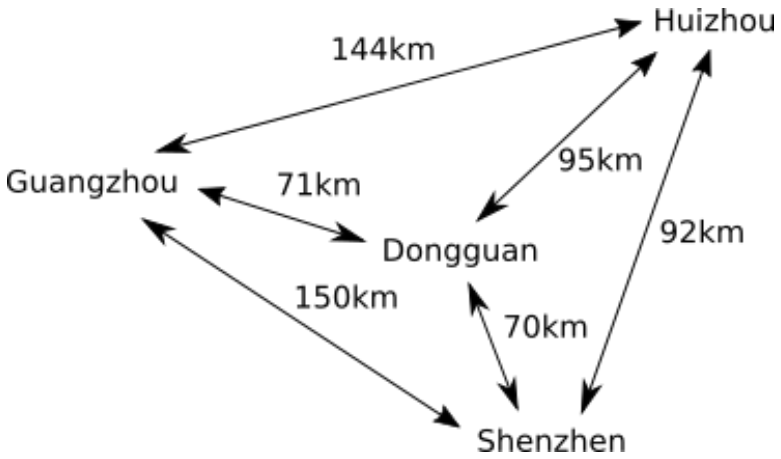
What is the route with the shortest distance to visit all the cities?




A naïve approach would be: computing all the possible routes and keep the one with the shortest total distance.

## Example: the travelling salesman problem (naïve approach)

What is the route with the shortest distance to visit all the cities?




A naïve approach would be: computing all the possible routes and keep the one with the shortest total distance.

### This approach is not suited when there are too many cities

- For 4 cities: 24 distance additions (few miliseconds)
- For 20 cities: $2 \times 10^{18}$ distance additions (more than 1 million years...)

## How can I know if my algorithm design is appropriate enough?

As mentioned before, by evaluating the complexity!

[3] Note that most of the time when people refer to *complexity* they mean time related complexity (known as *time complexity*)

## How can I know if my algorithm design is appropriate enough?

As mentioned before, by evaluating the complexity!

## Complexity

It gives the **number of fundamental operations** that will be processed by the algorithm according to the **amount of input data**. Example: the number of distance additions (see previous slide).

[3] Note that most of the time when people refer to *complexity* they mean time related complexity (known as *time complexity*)

## How can I know if my algorithm design is appropriate enough?

As mentioned before, by evaluating the complexity!

## Complexity

It gives the **number of fundamental operations** that will be processed by the algorithm according to the **amount of input data**. Example: the number of distance additions (see previous slide).

Knowing the complexity permits to...

- ... indirectly estimate the total time of execution
- ... indirectly estimate the total memory consumption[3]

[3] Note that most of the time when people refer to *complexity* they mean time related complexity (known as *time complexity*)

## Fundamental operation

Fundamental operations are the building-blocks of the algorithm[4]

A fundamental operation ...

- ... has a **well defined time and memory consumption**
  - Ex: value assignation of an addition, computing a surface, etc.
- ... **can be complex**
  - i.e: "absorbing" a bunch of smaller and various operations
  - Ex: algebra operations on an input vector (machine learning)

[4] Note that: two algorithms can only be compared if they have the same fundamental operations.

## Fundamental operation

Fundamental operations are the building-blocks of the algorithm[4]

A fundamental operation ...

- ... has a **well defined time and memory consumption**
  - Ex: value assignation of an addition, computing a surface, etc.
- ... **can be complex**
  - i.e: "absorbing" a bunch of smaller and various operations
  - Ex: algebra operations on an input vector (machine learning)

## What to do if there are several fundamental operations?

In case there are several fundamental operations (no further "absorption" or simplification is possible to have one unique fundamental operation): the final complexity will be the addition of all the complexities.

[4] Note that: two algorithms can only be compared if they have the same fundamental operations.

## Recall

The complexity is the **number of fundamental operations** that will be processed by the algorithm according to the **amount of input data**.

Example: looking for the value $x$ in the array of integer $T$ (of size $N$)

**Data:** $T$(sorted by increasing values)
$k = 0$;
**while** $k < N$ **do**
  **if** $T[k] >= x$ **then**
    return $k$;
  **else**
    $k = k + 1$;
  **end**
**end**

**Algorithm 3:** pseudo-code of basic search algorithm

- Here the fundamental operation is the conditional test "if"
- Complexity: at best 1, at worst $N$ and in average: $\frac{N}{2}$ operations

## Three complexity definitions

There are three complexity definitions: **complexity at best**, **complexity at worst** and **complexity in average**. The complexity at worst and in average are the most used (because there are pessimistic predictions).

- complexity at best = minimum number of required operations
- complexity at worst = maximum number of operations we could have
- complexity in average = most likely number of operations

complexity at best $<$ complexity in average $<$ complexity at worst

Example 2: looking for the value $x$ in the array of integer $T$ (of size $N$) with a dichotomic search

**Data:** $T$(sorted by increasing values)
$k = 0$; $i = 0$; $j = N$;
**while** $i < j$ **do**
  $k = (i + j)/2$;
  **if** $T[k] < x$ **then**
    $i = k + 1$;
  **else**
    $j = k$;
  **end**
**end**
**if** $i == N - 2$ *and* $T[i] < x$ **then**
  $i = N - 1$;
**end**
return $i$;

**Algorithm 4:** pseudo-code of dichotomic search algorithm

Here, the complexity at worst = $log_2 N$ (much better than the basic search!)

Indeed:

1. At each loop, N is divided by 2 and the smallest instance size is 1, so we have: $1 \leq \frac{N}{2^{number\ \ operations}}$
2. By multiplying by $2^{number\ \ operations}$ we get: $2^{number\ \ operations} \leq N$
3. By composing with *log* we get: $number\ \ operations \times log(2) \leq log(N)$
4. Finally we get: $number\ \ operations \leq log_2(N)$

## The big $O$ notation

The complexity is most of the time expressed with the big $O$ notation :

- $f = O(g) \Leftrightarrow$ f is *dominated* by g
- $f = O(g) \Leftrightarrow \exists n_o$ such that for all $n > n_o, f(n) \leq c \times g(n)$

The big $O$ notation is well suited for complexity analysis because it represents well the **order of magnitude** and the **evolution with respect to the size**.

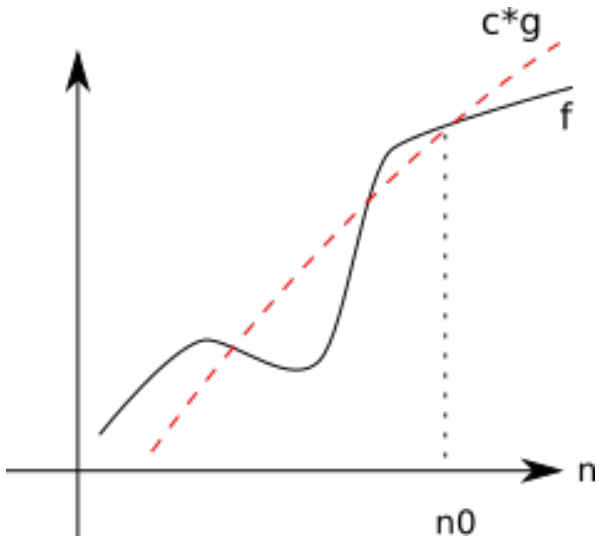



Figure: $f$ is dominated by $g$

big $O$ notations of complexities:

- $N \Rightarrow O(N)$
- $log_2(N) \Rightarrow O(log_2(N) \Rightarrow O(log(N))$
- $2 \times N^2 \Rightarrow O(N^2)$
- $5 \times N^3 + 200 \times N^2 + 15 \Rightarrow O(N^3)$
- $2 \times log_2(N) + 15 \times ln(N) \Rightarrow O(log(N))$
- $2^{64} \Rightarrow O(1)$
- $log(N^{10}) + 2\sqrt{N} \Rightarrow O(\sqrt{N})$
- $2^N + N^{1000} \Rightarrow O(2^N)$
- ...

Let's assume $f$ is our functional operation, then:

```
for i ← 0 to N do
  | f()
end
```

$\Longrightarrow O(N)$

```
for i ← 0 to N do
  | for j ← 0 to N do
  |   | f()
  | end
end
```

$\Longrightarrow O(N^2)$

```
for i ← 0 to N do
  | for j ← 0 to N do
  |   | for j ← 0 to N do
  |   |   | f()
  |   | end
  | end
end
```

$\Longrightarrow O(N^3)$

```
for i ← 0 to N do
  | for j ← 0 to M do
  |   | for j ← 0 to P do
  |   |   | f()
  |   | end
  | end
end
```

$\Longrightarrow O(N \times M \times P)$

Complexity examples of well-known algorithms:

- $O(1)$: accessing array index, inserting a node in linked list, pushing and popping (stack), insertion and removal from queue, etc.
- $O(n)$: traversing an array, traversing a linked list, linear search, delete element in a not sorted linked list, comparing two strings
- $O(log(N))$: dichotomy or binary search, finding largest/smallest number in a binary search tree, calculating Fibonacci Numbers, etc.
- $O(N \times log(N))$: merge sort, heap sort, quick sort, etc.
- $O(N^2)$: bubble sort, insertion sort, selection sort, traversing a simple 2D array, etc.

| Size | 1 | $log(N)$ (logarithmic) | N (linear) | $N \times log(N)$ (N-logarithmic) | $N^2$ (quadratic) | $N^3$ (cubic) | $2^N$ (exponential) |
|---|---|---|---|---|---|---|---|
| $10^2$ | $1\mu$s | $7\mu$s | 0.1ms | 0.7ms | 10ms | 1s | $4 \times 10^7$Gy |
| $10^3$ | $1\mu$s | $10\mu$s | 1ms | 10ms | 1ms | 17min | X |
| $10^4$ | $1\mu$s | $13\mu$s | 10ms | 130ms | 1.7ms | 12d | X |
| $10^5$ | $1\mu$s | $17\mu$s | 0.1s | 1.7s | 2.8h | 32y | X |
| $10^6$ | $1\mu$s | $20\mu$s | 1s | 20s | 12d | 32Ky | X |

| Time | 1 | $log(N)$ (logarithmic) | N (linear) | $N \times log(N)$ (N-logarithmic) | $N^2$ (quadratic) | $N^3$ (cubic) | $2^N$ (exponential) |
|---|---|---|---|---|---|---|---|
| 1s | X | X | 1M | 63K | 1K | 100 | 20 |
| 1min | X | X | 60M | 3M | 7K | 400 | 26 |
| 1h | X | X | 3.6G | 130M | 60K | 1.5K | 32 |
| 1d | X | X | 96G | 3G | 300K | 4.4K | 36 |

- 1 day with linear time complexity:
  - $96 \times 10^9$ fundamental operations are done
  - If 1 fundamental operation generates 1Byte, then total storage = 96GO
- 1 second with quadratic time complexity:
  - $1 \times 10^3$ fundamental operations are done
  - If 1 fundamental operation generates 1MO, then total storage = 1GO

| Time | 1 | $log(N)$ (logarithmic) | N (linear) | $N \times log(N)$ (N-logarithmic) | $N^2$ (quadratic) | $N^3$ (cubic) | $2^N$ (exponential) |
|---|---|---|---|---|---|---|---|
| 1s | X | X | 1M | 63K | 1K | 100 | 20 |
| 1min | X | X | 60M | 3M | 7K | 400 | 26 |
| 1h | X | X | 3.6G | 130M | 60K | 1.5K | 32 |
| 1d | X | X | 96G | 3G | 300K | 4.4K | 36 |

- 1 day with linear time complexity:
  - $96 \times 10^9$ fundamental operations are done
  - If 1 fundamental operation generates 1Byte, then total storage = 96GO
- 1 second with quadratic time complexity:
  - $1 \times 10^3$ fundamental operations are done
  - If 1 fundamental operation generates 1MO, then total storage = 1GO

**Time complexity, space complexity, what's the difference?**

Time complexity focuses on the **fundamental operation iterations**, space complexity on the **space consumption**.

1. Data
   - Data, types and properties
   - Units
2. Memory
   - Different memories for different usage
   - How it works?
3. Algorithm
   - What is an algorithm?
   - Example of algorithm
   - Well designing an algorithm is important!
   - Evaluating the complexity
   - Example of complexity calculus
   - Complexity definitions
   - The big O notation
   - Tips to find the complexity
   - Examples of time and memory consumption estimations
4. Conclusion

When designing your algorithm ...

- ...optimize well your **memory usage** if you need/can:
  - main memory is fast but small
  - mass memory is slow but big
  - limiting the number of mass memory accesses is good for performance
  - big intermediary results should be saved on mass memory for space
- ...optimize your **time complexity**:
  - a time complexity $< N^2$: we can process almost all input size
  - a time complexity between $N^2$ and $N^3$: intermediate input size
  - a time complexity $> N^3$: small input size
- ...don't forget to have a look to your **space complexity**:
  - it may not finish because the memory got completely saturated!

**Note**

A more powerful computer does not "improve the complexity". It only *reduces* the time spent on each fundamental operation. But the issue is the same.